



中山大學
SUN YAT-SEN UNIVERSITY

指针与字符串、函数

學大山中立國
中山大学计算机学院



讲课人：潘茂林

目录

CONTENTS

01

const 指针

02

字符串字面量

03

字符相关库函数

04

void 指针

05

函数指针



const 与 指针 / 常指针 - 申明

const 限定类型对象的左值表达式，表示不可修改左值，即不可赋值

1. **const** 限定关键字与指针变量

- 限定最终指向的对象或数组元素，**const**（红色）在类型前，例如：
 - ✓ `x, *p, **pp, p[]` 等都是 **const**
- 限定指针变量，**const** 在指针变量或引用的指针前面，例如：
 - ✓ `p, *pp, pp[]` 都是 **const**（紫色）
 - ✓ `pp` 是 **const**（黑色）

2. **const** 不宜出现在类型和 * 之间

- 例如：`int const *p;`
- 但 Dev C++ 中等价于 `const int *p;`

语法

const int x

const int * **const** p

const int * **const** * **const** pp

const 对象/变量

7

&x

&p



const 与 指针 / 常指针 - 应用示例

const 限定的左值表达式

1. 只能初始化，不能赋值
2. `const int` 和 `int` 是不同类型
 - 非const 可隐式转为 `const` 版本
 - `const` 赋予 非const 必须显式转型
3. 在函数声明中
 - `double x[const]` 等价 `double * const x`
 - `const double x[]` 等价 `const double *x`

```
/*const point*/
#include<stdio.h>

int main() {
    const int n = 1; // const 类型对象
    n = 2; // 错误: n 的类型为 const 限定

    int x = 2; // 无限定类型对象
    const int* p = &x;
    *p = 3; // 错误: 左值 *p 的类型为 const 限定
    int *const q = &x;
    *q = 3;
    q = NULL; // 错误: 指针 q 的类型为 const 限定

    int *p1 = 0;
    const int *a = p1; //OK, 非const 转 const
    p1 = a; // 错误: 舍弃限定符 ( const int 到 int )
    p1 = (int*)a; //必须显式转型

    return 0;
}
```



字符串字面量 - 常指针

字符串字面量在编译阶段都会存放在一个独立资源文件中，与代码一起加载到只读代码段。

1. 注意申明 `char *s` 和 `char t[N]` 有字面量初值的不同
 - `*s` 指向只读字面量
 - `t[]` 保存字面量的复制
2. 不能修改字符串字面量
 - `const char *` 申明字符串字面量可减少BUG
3. 相同字面量只有一份
`(char*)"STUDENT" == s`

```
/*const string*/  
...  
char *s = "STUDENT";  
// 这里等价 (char*)"STUDENT"  
// "STUDENT" 加载到代码段，只读  
char t[] = "STUDENT";  
// 等价 (char[]){ "STUDENT" }  
// 先开辟 t[8]，然后初始化，即  
// memcpy(t, (char*)"STUDENT", 8);  
  
*t = 'T';  
*s = 'T'; // 错误：段错误  
printf("%s,%s,%d\n", s, t,  
        (char*)"STUDENT" == s);
```



字符串字面量 - 数组

字符串字面量数组空间不连续，只能使用指针数组或指针的指针。每个字面量独立存放，与代码一起加载到只读代码段。

1. 注意申明 `char *s[]` 和 `char t[N][N]` 有字面量初值差异
 - `*ss[]` 指向只读字面量
 - `t[M][N]` 是连续的数组
2. 不能修改字符串字面量
 - 用 `const char *ss[]` 申明
3. 相同字面量只有一份，因此 `pp[0] == pp[1]`

```
/*const string*/
...
char *(ss[]) = {"hello", "hello", "!"};
char **pp = ss;
pp = (char *[]){ "hello", "hello", "!" };
pp[0][0] = 'H'; //错误: 段错误

char sa[][10] = {"hello", "hello", "!"};
sa[0][0] = 'H';

printf("%s,%s,%d\n", *pp, *sa,
        pp[0] == pp[1]);
```



案例研究 - 星期几?

给定 year, month, day, 请问是星期几?

```
/*weekday*/
const char* weekday(int year,int month, int day) {
    const char *weekdays[] = {"Sunday",
        "Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday", };
    static int m[12] = {31,28,31,30,31,30,31,31,30,31,30,31};
    m[1] = leap(year)? 29 : 28;
    int sum_days = day;
    for (int i=0,*p=m;i<month-1;i++)
        sum_days += *(p++);
    // printf("%d\n",sum_days);
    for (int i=1;i<year;i++)
        sum_days += leap(i)? 366 : 365;
    // printf("%d\n",sum_days);
    return weekdays[sum_days % 7];
}
```

问题:

- 为什么字符串字面量指针可作为函数返回
- 总结: 查表法, if else if, switch 的使用场景
- 请将函数分成几个段落, 给出确当的注释



字符串相关函数

空终止字节字符串（NTBS）是尾随零值字节（空终止字符）的非零字节序列。

头文件	分类	常见函数（红色要求掌握）
ctype.h	字符分类	isalnum, isalpha, isdigit, islower, isupper, isspace , isprint
	字符操作	tolower, toupper
stdlib.h	转换成数值	atof, atoi（注：itoa 是非标函数）
string.h	字符串操作	strcpy , strncpy , strcat , strncat
	字符串检验	strlen , strcmp , strchr , strrchr, strstr
	字符数组操作	memchr, memcmp, memset , memcpy , memmove
	杂项	strerror

参考：[空终止字节字符串 - cppreference.com](http://cppreference.com)



字符串库函数 - 参考实现

isspace 的一种实现。对于不连续的字符，使用数组与指针处理，很便捷。

```
/*pointer-str-func*/  
int isSpace(int ch) {  
    const char *p = " \t\n\v\f\r";  
    for (;*p;p++) {  
        if (ch==*p) break;  
    }  
    return *p;  
}
```

问题：

- 为什么在函数声明中不使用 char ch?
- 为什么不返回布尔值?

可能选项（多选）：

1. 历史原因
2. 内存变大了，省这点空间意义不大
3. 整数提升
4. 当年没布尔类型
5. 其他（请说明）



字符串库函数 - 参考实现

strcat 的一种实现。

```
/*pointer-str-func*/
char *StrCat( char *dest, const char *src ) {
    assert(dest && src); //src 或 dest 为空指针

    char *p = dest;
    for (;*p;p++);
    assert(!(src>=dest && src<=p)); //这个约束什么意思?

    for (;*src;src++,p++)
        *p = *src;
    *(++p)= 0;

    return dest;
}
```

问题:

- 请用一句话总结这个函数的每个段落
- 解释第二个 assert 含义。函数中使用 assert 对函数的调试有哪些好处
- 根据参考实现, 使用 strcat 有哪些约束, 例如:
 - dest,src 必须是NTBS
 - ...



void * 指针

void * 类型的指针变量仅保存指针的值（地址值），而忽略了指向数据的类型，因此编译器不允许使用 “*” 解引用运算，下标运算。“塞翁失马焉知非福”，

- 作为任意类型数据的指针，即任意类型指针都可以隐式转为 void *
- void * 类型的变量，可超越指针兼容性，隐式转为任意类型指针

[illegible]



void 指针- 动态存储应用

在 stdlib.h 申明了两个函数:

void* malloc(size_t size);

分配 size 字节的未初始化内存

void free(**void*** ptr);

回收malloc等分配的内存

1. p 和 q 都指向申请的内存

- p 指向二维数组
- q 指向一维数组
- 尽管p,q类型不兼容, 但程序无warning (但很容易导致BUG)

2. malloc 和 free 必须成对

- 分配的指针多次free行为未定义;
- 分配的指针不回收, 会导致内存溢出

```
/*void-malloc-free*/
#include<stdio.h>
#include<stdlib.h>

int main() {
    int m=3,n=4;
    int *q = malloc(m*n*sizeof(int));
    void *t = q;
    int (*p)[n] = t;
    for (int i=0;i<m*n;i++)
        q[i] = i+1;
    for (int i=0;i<m;i++) {
        for (int j=0;j<n;j++)
            printf("%4d",p[i][j]);
        printf("\n");
    }
    free(q);
}
```



void 指针-函数泛类型参数

1. MemRepeat

- 用同类型的变量值，填充 dest 数组，例如：

```
int a[10], d=7;  
MemRepeat(a,&d,sizeof(d),10);
```

2. Printx

- 打印 p 指向对象的二进制值，例如
- ```
double f=3;
Printx(&f,sizeof(f))
```

```
/*void-parameter*/
void* MemRepeat(void *dest,void *src,size_t
src_sz,size_t count) {
 assert(dest && src);
 void *p=dest;
 for (int i=0;i<count;i++) {
 memcpy(p,src,src_sz);
 p += src_sz;
 src += src_sz;
 }
 return dest;
}

void Printx(void *p, size_t bytes_count) {
 assert(p);
 for (int i=0;i<bytes_count;i++,p++)
 printf("%4x", *(unsigned char*)p);
 printf("\n");
}
```



## 函数指针

程序中定义了一个函数，在编译时系统就会为这个函数代码分配一段存储空间，这段存储空间的首地址称为这个函数的地址。函数名表示的就是这个地址。

函数指针变量，简称函数指针，可用于存放函数**参数类型、数量**，及**返回值类型**与定义一致的函数地址。

函数指针定义：

函数返回值类型 (\* 指针变量名) (函数参数类型列表);

如：

```
int(* p)(int, int);
```

注意：函数指针变量**没有算术运算，下标运算**。



## 函数指针 - 定义与解引用

```
1 #include <stdio.h>
2
3 int Max(int a,int b) {
4 if (a>b)
5 return a;
6 else
7 return b;
8 }
9
10 int main() {
11 int a = 5, b = 6;
12 int (*p)(int, int);
13 p = Max;
14 int c = (*p)(a, b);
15 printf("%d is bigger.", c);
16 return 0;
17 }
```

6 is bigger.

具体的使用如下:

|                                |                             |
|--------------------------------|-----------------------------|
| <code>int Func(int x);</code>  | <code>/*声明一个函数*/</code>     |
| <code>int (*p) (int x);</code> | <code>/*定义一个函数指针*/</code>   |
| <code>p = Func;</code>         | <code>/*将Func函数的首地址赋</code> |
| <code>给指针变量p*/</code>          |                             |



## 函数指针 - 回调函数

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 const int MAXN = 5;
5
6 int cmpfunc (const void * a, const void * b) {
7 return (*(int*)a - *(int*)b);
8 }
9
10 int main() {
11 int n = 5;
12 int arr[MAXN] = {5, 3, 1, 2, 4};
13 qsort(arr, 5, sizeof(int), cmpfunc);
14 printf("排序之后的列表: \n");
15 for (int i=0; i<n; i++)
16 printf("%d ", arr[i]);
17 }
```

排序之后的列表:

1 2 3 4 5

那么qsort是怎么知道如何排列的呢?  
这就要靠cmpfunc了。

如果我们想从大到小排列，只需要  
修改cmp废话c即可。

实际上，cmpfunc就是一个回调函数！





## 函数指针 - 回调函数

```
void qsort(void *base,int nelem,int width,
 int (*fcmp)(const void *,const void *));
int fcmp(const void *,const void *);
```

调用函数有两种方法：

1. 直接调用：在函数A的函数体里通过书写函数B的函数名来调用之，使内存中对应函数B的代码得以执行。
2. 间接调用：在函数A的函数体里并不出现函数B的函数名，而是使用指向函数B的函数指针来使内存中属于函数B的代码片断得以执行。

**回调函数**，函数指针作为某个函数的参数，由函数外部逻辑决定函数行为。例如，排序时把数组两个元素的比较逻辑交给外部传入的函数。



## 选择排序 - 函数回调版本

```
/*selected sort callback version*/
void SelectedSortInt(int *nums, int n, int (*fcmp)(int,int)) {
 for (int i=0; i<=n-2; i++) {
 //find minimum idx
 int idx=i;
 for (int j=i+1; j<=n-1; j++)
 if ((*fcmp)(nums[j],nums[idx])<0)
 idx = j;
 //swap
 if (i!=idx) {
 int temp = nums[i];
 nums[i] = nums[idx];
 nums[idx] = temp;
 }
 }
}
```



中山大學  
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



编制人：课题组